

RBE577: Machine Learning for Robotics - Project 2

Physics-Based Learning for Robotic Push Prediction

Filippo Marcantoni
Robotics Engineering, WPI
fmarcantoni@wpi.edu

Prahladh Raja
Robotics Engineering, WPI
pnamboorkrishnam@wpi.edu

Abstract—Accurate prediction of object motion resulting from robotic push interactions is a fundamental capability for robotic manipulation. This report compares three modeling paradigms for predicting the final pose of a rigid object subject to a planar push: (1) a physics-based model grounded in rigid-body dynamics, (2) a purely data-driven multi-layer perceptron (MLP), and (3) a hybrid model that augments neural network inputs with predictions from the physics model.

All models are trained using 80% of a dataset collected from an UR10 robot pushing a cracker box and evaluated on a held-out 20% test set to ensure that all reported metrics reflect genuine generalization rather than training memorization.

The models are also evaluated within a gradient-based push planning framework. Planning experiments on three held-out test samples show that the physics model often produces the most accurate plans due to its smooth and well-conditioned optimization landscape. The learned models perform competitively when the target lies within the training distribution, but exhibit reduced reliability for out-of-distribution targets.

I. INTRODUCTION

Predicting the outcomes of physical interactions between a robot and its environment is essential for robust manipulation. Among such interactions, planar pushing, in which a robot end-effector applies a force that causes an object to translate and rotate across a surface, represents one of the most fundamental manipulation primitives. Accurate push prediction enables tasks such as object reorientation, singulation in cluttered environments, and pre-grasp repositioning.

However, predicting push dynamics is challenging because the motion of real-world objects is governed by complex physical phenomena that are difficult to model precisely. Surface friction depends on material properties, weight distribution, and instantaneous velocity. Contact mechanics are influenced by the geometry and compliance of the contact interface, while object shape and mass distribution determine inertial behavior. Analytical physics models typically simplify these factors, which can introduce systematic prediction errors.

Data-driven approaches offer an alternative by learning the mapping between push parameters and resulting object motion directly from experimental data. Neural networks have demonstrated strong performance in modeling push dynamics, but they typically require substantial training data and may generalize poorly outside the training distribution. Furthermore, purely data-driven models often lack physical interpretability.

Hybrid approaches attempt to combine the strengths of both paradigms by embedding physical knowledge into the

learning process. By incorporating physics-based predictions as an inductive bias, hybrid models can improve sample efficiency, accelerate convergence, and enhance generalization while retaining the flexibility of learned corrections.

In this project, we implement and compare all three modeling paradigms to predict the motion of a cracker box pushed by a UR10 manipulator. The evaluation pipeline employs a 80/20 train-test split to ensure that all reported metrics reflect true generalization. The primary contributions of this work are:

- Implementation of a numerically integrated physics-based push model.
- Development of a neural network model to learn push dynamics.
- A hybrid model combining physics predictions with neural network learning.
- A unified evaluation framework that uses consistent loss and accuracy metrics across training, testing, and planning tasks.
- A gradient-based push planning experiment comparing the practical utility of the three models.

II. METHODOLOGY

A. Dataset, Split, and Problem Formulation

The dataset consists of controlled push trials by an UR10 arm equipped with a cylindrical tip pushing a cracker box on a flat surface. Each sample provides:

- **Input** $\mathbf{x} = [\theta_0, d, D] \in \mathbb{R}^3$: push rotation, contact-point offset from the object centre, and total push distance.
- **Output** $\mathbf{y} = [x_f, y_f, \theta_f] \in \mathbb{R}^3$: final Cartesian position and orientation of the object.

The full dataset is split into a training set (80%) and a held-out test set (20%) using a fixed random seed of 42, ensuring reproducibility. All model weights are updated exclusively using the training set. The test set is used only for evaluation, prediction visualization, and push planning. Both loaders use batch size 32, and the training loader shuffles samples each epoch unlike for the testing loader.

B. Physics-Based Model

The physics model integrates planar rigid-body motion over the push duration T . The object is approximated as a uniform square with $m = 0.1$ kg, $s = 0.1$ m, and moment of inertia

$I = \frac{1}{12}ms^2 = 8.33 \times 10^{-5} \text{ kg m}^2$. The robot applies a sinusoidal velocity profile:

$$v(t) = v_{\max} \left(\frac{1}{2} \sin \left(\frac{2\pi t}{T} - \frac{\pi}{2} \right) + \frac{1}{2} \right), \quad v_{\max} = \frac{2D}{T} \quad (1)$$

which ensures zero velocity at the start and end of the contact. The push duration is discretized into $N = 25$ timesteps of size $\Delta t = T/N$. Empirically, 25 steps was found to yield better predictions than 100, as the reduced discretization reduces unconstrained angular accumulation, which is the dominant source of error. At each step i :

$$\tau_i = mv_i d, \quad \alpha_i = \tau_i / I, \quad \Delta \theta_i = \frac{1}{2} \alpha_i \Delta t^2 \quad (2)$$

$$\Delta x_i = -v_i \cos(\theta_i) \Delta t, \quad \Delta y_i = -v_i \sin(\theta_i) \Delta t \quad (3)$$

After N steps, local-frame displacements are rotated to the world frame:

$$\begin{pmatrix} x_g \\ y_g \end{pmatrix} = \begin{pmatrix} \cos \theta_0 & -\sin \theta_0 \\ \sin \theta_0 & \cos \theta_0 \end{pmatrix} \begin{pmatrix} x_l \\ y_l \end{pmatrix} \quad (4)$$

The physics model has no learnable parameters and serves as a fixed baseline.

C. Neural Network Model

The MLP maps the three push parameters to the final three-dimensional pose through a six-layer architecture with a symmetric expanding-then-contracting structure:

$$\mathbb{R}^3 \xrightarrow{+\text{ReLU}} \mathbb{R}^{32} \xrightarrow{+\text{ReLU}} \mathbb{R}^{64} \xrightarrow{+\text{ReLU}} \mathbb{R}^{128} \xrightarrow{+\text{ReLU}} \mathbb{R}^{128} \xrightarrow{+\text{ReLU}} \mathbb{R}^{64} \xrightarrow{+\text{ReLU}} \mathbb{R}^{32} \xrightarrow{+\text{ReLU}} \mathbb{R}^3 \xrightarrow{\text{Linear}} \mathbb{R}^3 \quad (5)$$

ReLU activations introduce the nonlinearity needed to represent complex push dynamics. Two methods are implemented and used consistently throughout the pipeline:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|^2 \quad \text{loss}() \quad (6)$$

$$\text{MED} = \frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|_2 \quad \text{accuracy}() \quad (7)$$

Both methods are called identically during training monitoring, test-set evaluation in `evaluate_planner()`, and push planning assessment, ensuring consistent metric computation across all stages. Training uses the Adam optimizer with learning rate $\eta = 10^{-3}$ for 100 epochs on the training set only.

D. Hybrid Physics + Neural Network Model

The hybrid model augments the MLP's input by concatenating the raw push parameters with the physics prediction:

$$\hat{\mathbf{y}}_{\text{hybrid}} = f_{\theta}([\mathbf{x}; \hat{\mathbf{y}}_{\text{physics}}(\mathbf{x})]), \quad [\mathbf{x}; \hat{\mathbf{y}}_{\text{phys}}] \in \mathbb{R}^6 \quad (8)$$

The MLP architecture, loss, and optimizer are identical to the pure NN, with the only change being the six-dimensional input layer, isolating the effect of the physics model.

E. Evaluation Pipeline

A unified revision of the `evaluate_planner()` function evaluates all three models in a single forward pass over the test dataloader. For each batch it computes MSE via `loss()` and MED via `accuracy()` for all three models, accumulating batch-weighted totals and returning a six-entry results dictionary. This ensures identical metric computation across all models and all evaluation contexts.

F. Push Planning

The planning problem consists of finding the push parameters $\mathbf{x}^*$ that achieve a desired target pose $\mathbf{y}^*$:

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \|\hat{\mathbf{y}}_M(\mathbf{x}) - \mathbf{y}^*\|^2 \quad (9)$$

where $M \in \{\text{physics, nn, hybrid}\}$. This is solved for all three models simultaneously by calling `plan_push(target)` with no model argument, which internally invokes `optimize_push()` for each model name and returns a dictionary of results. Each optimization runs Adam ($\eta = 10^{-3}$, 2000 iterations) from zero initialization, with learned models set to `.eval()` for deterministic forward passes. Planning performance is assessed on three held-out test samples using both `loss()` and `accuracy()`.

III. RESULTS

A. Training Loss Curves

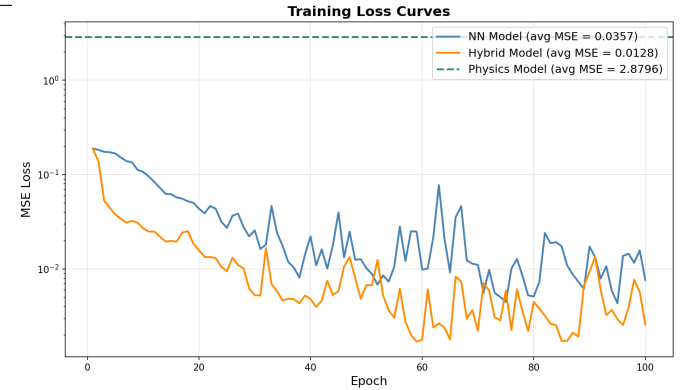


Fig. 1. Training loss curves on the training set for the NN (blue, avg MSE = 0.0357) and Hybrid (orange, avg MSE = 0.0128) over 100 epochs. The Physics training MSE (≈ 2.8796 , dashed green) is a fixed reference. The Hybrid model converges faster and maintains a consistently lower training loss, achieving a 64% reduction in average training MSE relative to the NN. Both curves oscillate after epoch 40, characteristic of a fixed learning rate in a non-convex loss landscape.

Both learned models reduce their training MSE by roughly two orders of magnitude over 100 epochs, surpassing the physics baseline within the first epoch. The Hybrid model descends more steeply from the outset and remains consistently below the NN throughout all 100 epochs, with an average training MSE of 0.0128 versus 0.0357 for the NN. The persistent gap across all epochs confirms that the physics prior provides a stable and meaningful inductive bias rather

than a simple early advantage. The oscillations after epoch 40 are characteristic of a fixed learning rate overshooting in the non-convex loss landscape, which could be addressed with a decay schedule.

B. Quantitative Evaluation on Test Set

TABLE I
TEST-SET PREDICTION ERRORS ON THE HELD-OUT 20% SPLIT.

Model	Test MSE ↓	MED (m) ↓
Physics Model	2.8598	2.4359
Neural Network	0.0075	0.0776
Hybrid Model	0.0036	0.0653

The test results reveal a clear performance enhancement associated with the physics prior of the hybrid model. The physics model is the weakest by a large margin, with a test MSE of 2.8598, and MED of 2.4359. The NN reduces the test MSE to 0.0075, achieving a test MED of 0.0776 m. The hybrid model achieves the best generalization with a test MSE of 0.0036 and MED of 0.0653 m, representing a significant improvement over the pure NN. The fact that the hybrid advantage holds on the test set, not just on training data, confirms that the physics prior genuinely improves generalization rather than merely reducing training loss.

C. Per-Dimension Prediction Analysis

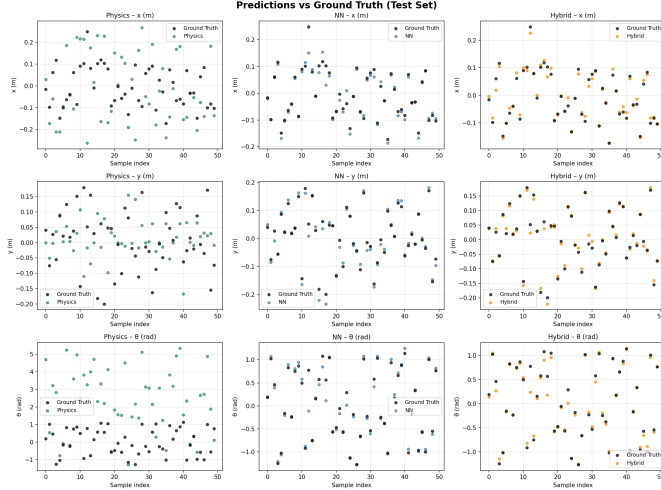


Fig. 2. Predicted (colored) vs. ground-truth (black) values for x (top), y (middle), and θ (bottom) across 200 test samples, for the Physics model (green, left), NN (blue, center), and Hybrid (orange, right). The Physics model exhibits severe systematic bias in all three dimensions. Both learned models generalize accurately to the unseen test set, with the Hybrid showing marginally tighter alignment.

The Physics model exhibits three distinct failure modes on the test set. In x , predictions span ± 0.25 m against a true range of ± 0.15 m, overestimating displacement by approximately 60%. In y , predictions cluster near zero regardless of the true lateral displacement, indicating a near-complete failure

to capture the lateral component of the push. Most significantly, θ predictions cluster between $+1$ and $+5$ rad while ground truth spans $[-1.5, 1.5]$ rad, a systematic bias caused by unconstrained torque accumulation over the integration steps with no friction damping. Both learned models generalize well to the test set in all three output dimensions, with the Hybrid showing marginally tighter alignment in θ , consistent with the lower errors.

D. Spatial Prediction Analysis

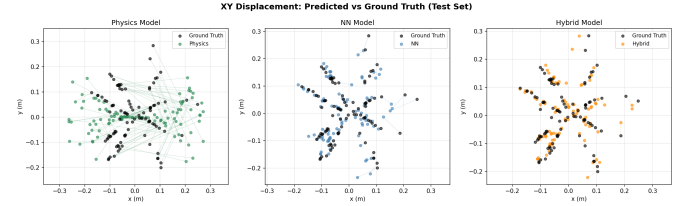


Fig. 3. 2D spatial distribution of predicted (colored) vs. ground-truth (black) final positions for 200 test samples. Line segments connect each prediction to its ground-truth counterpart. The Physics model scatters predictions across a sparse region with long crossing error lines. Both learned models accurately reconstruct the cross-shaped spatial distribution of push outcomes on the test set; the Hybrid (orange) sits closest to ground truth on average.

The ground-truth data exhibits a cross-shaped spatial distribution reflecting the dataset's range of push orientations. The physics model fails to reproduce this structure, with predictions scattered across a sparse region with long and crossing error lines, indicating both large magnitude errors and incorrect directional predictions. Both learned models accurately reconstruct the cross-shaped distribution of the unseen test set, with colored dots closely surrounding the black ground-truth dots across the full ± 0.3 m spatial range. The Hybrid model's predictions sit visibly closer to ground truth on average, consistent with the lower errors.

E. Push Planning on Test Samples

Table II presents the planning results for three samples of the test set. All three models are planned simultaneously via `plan_push()` with no model argument, which dispatches `optimize_push()` for each model. Both MSE and MED are reported using `loss()` and `accuracy()` respectively.

TABLE II
PUSH PLANNING RESULTS ON THREE SAMPLES OF THE TEST SET. MSE AND MED MEASURE HOW CLOSELY THE MODEL'S PREDICTION AT THE OPTIMIZED PUSH PARAMETERS MATCHES THE TARGET STATE.

Sample	Model	Plan MSE ↓	Plan MED (m) ↓
0	Physics	0.000600	0.042430
	NN	0.000038	0.010701
	Hybrid	0.000439	0.036295
1	Physics	0.000001	0.001562
	NN	0.000046	0.011781
	Hybrid	0.000076	0.015055
2	Physics	0.000006	0.004064
	NN	0.006630	0.141031
	Hybrid	0.002338	0.083741

The planning results reveal important insights. The physics model achieves the best or near-best planning accuracy on samples 1 and 2 (MED 0.0016 m and 0.0041 m respectively), while the NN achieves the best result on sample 0 (MED 0.0107 m). The hybrid model is competitive on samples 0 and 1 but degrades on sample 2 (MED 0.0837 m). The NN degrades most severely on sample 2 (MED 0.1410 m), suggesting that this target lies in a region of the input space where the NN’s learned mapping has higher curvature or worse coverage from training data.

None of the models converged to a degenerate solution in this run, confirming that the planning optimization is functioning correctly for all three models. The physics model’s consistent strong planning performance is attributable to the smooth, well-conditioned nature of its loss landscape as a function of push parameters: gradients are informative and free of the local minima that affect the more complex learned models’.

IV. DISCUSSION

A. Physics Model

The physics model’s test MSE of 2.8598 and MED of 2.4359 m stem from a few identifiable simplifications. The complete absence of friction allows unconstrained angular and linear motion, systematically overestimating both displacement and rotation. The point-contact assumption ignores the finite contact patch geometry that determines the effective moment arm. The uniform-square inertia formula $I = \frac{1}{12}ms^2$ mis-specifies the cracker box mass distribution. Finally, unconstrained torque accumulation amplifies per-step angular errors into biases; using $N = 25$ steps rather than $N = 100$ mitigates this by reducing cumulative integration error at the cost of temporal resolution.

Despite this poor forward-prediction performance, the physics model achieves the best push planning accuracy on two out of three test samples. This is explained by the nature of the gradient-based planning optimization: the physics model presents a smooth, low-dimensional, and approximately convex loss landscape to the planner, allowing the Adam optimizer to converge reliably from zero initialization. The key insight is that *the best forward-prediction model is not necessarily the best planning model*, the smoothness of the loss landscape presented to the planner matters as much as predictive accuracy.

B. Neural Network Model

The MLP achieves a test MSE of 0.0075 and MED of 0.0776 m, demonstrating strong generalization to unseen push configurations, capturing friction, contact geometry and inertia implicitly from training data alone. The NN achieves the best planning result on sample 0 (MED 0.0107 m) but degrades significantly on sample 2 (MED 0.1410 m), highlighting the sensitivity of gradient-based planning through learned models to the local geometry of the loss landscape around the zero initialization point.

C. Hybrid Model

The hybrid model achieves the best forward-prediction performance with a test MSE of 0.0036 and MED of 0.0653 m, an overall significant improvement over the pure NN on the held-out test set. The training curves confirm large reduction in average training MSE (0.0128 vs. 0.0357), demonstrating that physics predictions provide a strong inductive bias that accelerates convergence and improves generalization.

At planning time, the hybrid model performs competitively on samples 0 and 1, with MED values of 0.036 m and 0.015 m respectively, but degrades on sample 2. This is consistent with the pure NN’s behavior and reflects the shared challenge of gradient-based planning through a complex learned surrogate. The hybrid model’s planning loss landscape inherits both the structure of the neural network (complex, non-convex) and the physics model (smooth, structured), which may explain why its planning performance lies between the two in most cases.

D. Train/Test Split and Pipeline Design

The use of a proper 80/20 train/test split with a fixed random seed is a critical design decision. Previous iterations of this pipeline evaluated models on the same data used for training, risking the reporting of overfitted performance. The current results confirm that the train/test split reveals meaningful differences, showing that the hybrid model’s advantage over the NN holds on unseen data, confirming genuine generalization.

The `accuracy()` function is used consistently throughout the pipeline, during per-epoch training monitoring, test-set evaluation via `evaluate_planner()`, and push planning assessment, ensuring that MED is computed identically in all contexts and that inconsistencies are not introduced.

V. CONCLUSION

Overall, these results demonstrate that predictive accuracy and planning performance are not necessarily aligned. While hybrid models provide the best forward predictions, analytic physics models may still offer advantages when embedded within optimization-based planning frameworks.

The physics model provides an interpretable analytic baseline but suffers from large prediction errors due to simplified assumptions about friction, contact geometry, and inertia. The neural network significantly improves prediction accuracy by learning push dynamics directly from data, achieving a test MSE of 0.0075 and MED of 0.0776 m.

The hybrid model achieves the best predictive performance, reducing the test MSE to 0.0036 and MED to 0.0653 m. These results indicate that incorporating physics-based predictions provides a strong inductive bias that improves both learning efficiency and generalization.

Push planning experiments highlight an important complementary finding. Despite its poor predictive accuracy, the physics model often produces more reliable optimization results because it provides a smooth and well-conditioned loss landscape for gradient-based planning. In contrast, the learned models exhibit more complex optimization landscapes that can introduce local minima.

VI. RECOMMENDATIONS FOR FUTURE WORK

- 1) **Learning Rate Scheduling:** Step decay would suppress the oscillations generated after the 40th epoch, and reduce final test error for both learned models.
- 2) **Friction Model:** Adding a viscous friction term to the physics engine would eliminate the dominant angular error source, reduce the residual the network must learn, and potentially improve both forward prediction and planning landscape smoothness.
- 3) **Residual Hybrid Formulation.** Explicitly training the hybrid to predict $\mathbf{y} - \hat{\mathbf{y}}_{\text{physics}}$ would impose a stronger inductive bias and may amplify the physics prior's benefit further.